

Fondamenti di Computer Grafica

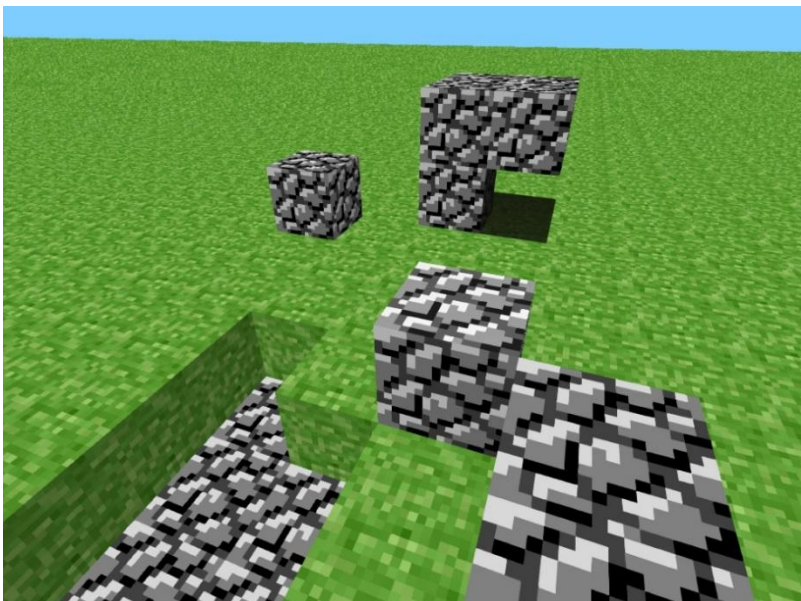
Relazione Progetto “NicoCraft”

Nicolò Bonifacio - 6176040

Introduzione

Questa relazione comprenderà i vari stadi di sviluppo, in dettaglio, del progetto di fondamenti di computer grafica “NicoCraft”. Un clone semplificato del gioco più venduto al mondo “Minecraft”. Minecraft ha molte e diverse “features” quindi per limiti di tempo e per l’ambito di questo progetto mi sono basato sulle vecchie versioni, ovvero quelle del lontano 2009-2010.

Esempio della versione PRE-CLASSIC (2009):



Si nota che è stata scelta questa versione di Minecraft per le funzionalità semplici e più raggiungibili per un progetto di questa scala.

In questa versione non sono presenti alcune delle meccaniche ritrovate nel videogioco moderno come la sopravvivenza o il combattimento. Qui invece vediamo esclusivamente la simulazione di un mondo digitale a Voxel 3D con l’aggiunta di meccaniche di movimento, rottura e piazzamento di blocchi (3D Voxels).

Obiettivi

Come **obiettivi** del progetto ho pensato di implementare, tramite l’utilizzo di OpenGL/SFML e la libreria GLM:

- Simulazione del Mondo Voxel-based in 3D (Quindi un mondo simulato o piccolo motore grafico).
- Presenza di camera che interagisca in prima persona con il mondo.
- Cubi o materiali di diversi tipi all’interno del mondo di gioco.
- Possibilità di piazzare diversi blocchi nell’area giocabile del mondo.
- Possibilità di rompere diversi blocchi nell’area giocabile del mondo.
- Aggiunta di blocchi trasparenti (Vedi attraverso il blocco le facce degli altri).
- Simulazione della luce Semplice tramite l’utilizzo di shader.

Dopo aver inquadrato gli obiettivi ho iniziato con la prima tappa. (Tappa01)

TAPPA01 – Generazione Cubo 3D composto da Vertici con Texture

Obiettivo:

La Tappa01 ha un obiettivo semplice, la generazione di un cubo 3D con la stessa texture su tutte le facce.

Come base della tappa si è usato il laboratorio 7 (Paradigma Camera-in-hand stile FPS).

Il laboratorio sette è stato molto ridotto, rimuovendo tutta la parte delle luci e modelli .off che erano presenti sia all'interno del mondo che nel codice e successivamente utilizzato per fare tutto il resto. La maggior parte dei file ".hh" sono stati rimossi perché non utili, però due file "Hotshaders.hh" e "Matrices.hh" sono stati mantenuti anche fino alla fine del progetto!

- Hotshaders per la parte di gestione degli shaders, compilazione, load, Linking, etc.
- Matrices invece è stata utile per il calcolo e utilizzo di matrici , traslazione, scaling , rotazione.

La logica di movimento della camera è la simile a quella utilizzata all'interno del laboratorio 7.

Principali Differenze:

1. GEOMETRIA: MESH CARICATA DA FILE → CUBO HARDCODED:

Rimosso il caricamento,renderizzazione e gestione delle mesh .off e rimpiazzato con la creazione di un cubo 3D di vertici semplice tramite l'utilizzo di un array statico.

```
static const float vertices[] = {  
    // Front (+Z)  
    -0.5f,-0.5f, 0.5f, 0,0,  
    0.5f,-0.5f, 0.5f, 1,0,  
    0.5f, 0.5f, 0.5f, 1,1,  
    -0.5f, 0.5f, 0.5f, 0,1,  
    -0.5f,-0.5f, 0.5f, 0,0,  
  
    // Back (-Z)
```

L'array è composto dalle coordinate dei vertici X,Y,Z e da due float aggiuntivi che sono le coordinate UV utilizzate dalla texture di quella faccia.

I primi 3 float (X, Y, Z) sono la posizione del vertice nello spazio 3D (in coordinate locali dell'oggetto).

Gli ultimi 2 float (U, V) sono le coordinate texture di quel vertice, che indicano quale punto della texture 2D deve essere "attaccato" a quel vertice.

2. ILLUMINAZIONE → TEXTURIZZAZIONE

Nella Tappa01 tutta la parte dell'illuminazione è stata rimossa ed eliminata.

L'illuminazione dinamica è stata sostituita da un fattore **costante** ambient_light = 0.9 trovato nello shader "shader_flat.frag". Un solo shader fisso, nessun hot-reload/switch tra shader diversi.

In più è stato aggiunto il texturing, implementato utilizzando "sf::Image image".

Libreria di SFML: #include <SFML/Graphics/Image.hpp>

Qui vediamo come la texture viene caricata e in caso viene utilizzata la texture di riserva (fallback).

```
if (!loaded) {  
    std::cerr << "Warning: Failed to load texture: " << path  
    << " - using fallback texture." << std::endl;  
  
    std::string fallbackPath = res + "missingTextureBlock.png";  
    if (!image.loadFromFile(fallbackPath)) {  
        std::cerr << "Error: Failed to load fallback texture: " << fallbackPath << std::endl;  
        exit(1);  
    }  
}
```

Qui avviene il binding della texture caricata da file:

```
auto size = image.getSize ();

glGenTextures (1, &texture);
glBindTexture (GL_TEXTURE_2D, texture);

glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

glTexImage2D (GL_TEXTURE_2D, 0, GL_RGBA, size.x, size.y, 0,
             GL_RGBA, GL_UNSIGNED_BYTE, image.getPixelsPtr ());
```

La funzione `glTexParameteri` indica invece a OpenGL come gestire la texture, da notare come `GL_NEAREST` sia stato scelto per il fatto che mantiene i pixel della texture netti e squadrati (pixel art) invece di sfumarli.

`GL_RGBA` invece sarà utile per texture trasparenti come vetro o foglie.

3. MODIFICHE ALLE SHADERS

In “**shader_flat.frag**” e “**shader_flat.vert**” prima trovavamo il modello di illuminazione Phong/flat con normali calcolate via derivate schermo ($dFdx/dFdy$), luce diretta, ambientale, specular, materiali.

Nella Tappa01 tutto questo è stato **sostituito** da un semplice campionamento texture e un fattore di luce ambientale fissa.

shader_flat.vert è stato modificato e rimpiazzato con questo:

```
#version 410 core

layout(location = 0) in vec3 VertexPosition;
layout(location = 1) in vec2 uvCoordinates;

uniform mat4 model;
uniform mat4 vp;

out vec2 interpolated_uv;

void main()
{
    gl_Position = vp * model * vec4 (VertexPosition, 1.0);
    interpolated_uv = uvCoordinates;
}
```

Dove il vertex shader (*shader_flat.vert*) riceve per ogni vertice:

- La posizione locale “**VertexPosition**” è la posizione del vertice nello spazio locale (X,Y,Z).
- Le “**uvCoordinates**” sono le coordinate **normalizzate** della texture [0.0-1.0]x[0.0-1.0].

Qui il vertex shader trasforma la posizione da spazio locale a spazio schermo tramite la matrice **model** (posizionamento nel mondo) e **vp** (view-projection), e passa le coordinate UV al fragment shader (*shader_flat.frag*) come output (`interpolated_uv`).

shader_flat.frag è stato modificato e rimpiazzato con questo:

```
#version 410 core

uniform sampler2D block_texture;

// luce di base fissa: nessun calcolo dinamico, solo un fattore di illuminazione costante
const float ambient_light = 0.9;

in vec2 interpolated_uv;

out vec4 fragment_color;

void main()
{
    vec3 tex_color = texture (block_texture, interpolated_uv).rgb;
    fragment_color = vec4 (tex_color * ambient_light, 1.0);
}
```

Il fragment shader riceve le UV interpolate e campiona il colore dalla texture bindata (block_texture) tramite la funzione texture().

Il colore ottenuto viene poi moltiplicato per un fattore costante (ambient_light = 0.9), che simula una leggera ombreggiatura uniforme, sostituendo così ogni calcolo dinamico di illuminazione presente nel laboratorio (**niente** più normali, **niente** più uniform light.*/material.*).

4. MODIFICHE AL MOVIMENTO

Nel laboratorio 7 la camera era gestita tramite **fcg::Trackball** (rotazione orbitale), oppure tramite una camera "volo libero" con movimento dolly su un solo asse.

Nella Tappa01 la camera è stata riprogettata come **camera FPS in prima persona stile Minecraft**, con due componenti distinte:

- **Rotazione** (mouse look):

```
void Look(float x, float y){
    if (!looking)
        return;

    float dx = x - lastMouseX;
    float dy = y - lastMouseY;
    lastMouseX = x;
    lastMouseY = y;

    phiDeg += dx * mouseSensitivity;
    thetaDeg += dy * mouseSensitivity;
    thetaDeg = thetaDeg > 90.0f ? 90.0f : thetaDeg;
    thetaDeg = thetaDeg < -90.0f ? -90.0f : thetaDeg;

    ViewProjection ();
}
```

- **Movimento** (tastiera):

```
void Move (float dt){
    float phiRad = glm::radians (phiDeg);
    glm::vec3 forward = { glm::sin (phiRad), 0.0f, -glm::cos (phiRad) };
    glm::vec3 right = { glm::cos (phiRad), 0.0f, glm::sin (phiRad) };
    glm::vec3 up = { 0.0f, 1.0f, 0.0f };
    glm::vec3 moveDir = {0.0f, 0.0f, 0.0f};
    for(const auto& keyBinding : moveBindings) {
        if (sf::Keyboard::isKeyPressed (keyBinding.key)) {
            moveDir +=
                keyBinding.direction.x * right +
                keyBinding.direction.y * up +
                keyBinding.direction.z * forward;
        }
    }
    if (glm::length(moveDir) < 0.0001f) return;
    moveDir = glm::normalize (moveDir);
    cameraPos += moveDir * moveSpeed * dt;

    ViewProjection ();
}
```

La differenza fondamentale rispetto al laboratorio 7 è che i vettori **forward** e **right** sono calcolati **usando solo phiDeg** [yaw], **ignorando** completamente **thetaDeg** [pitch]:

Questo significa che muoversi con [WASD] sposta sempre la camera sul piano orizzontale, indipendentemente da dove si sta guardando verticalmente — esattamente il comportamento di Minecraft Pre-classic.

TAPPA02 – Creazione del concetto e dei tipi di blocco

Obiettivo:

La Tappa02 ha come obiettivo l'introduzione di un sistema tramite il quale è possibile gestire i blocchi. Invece di andare ad applicare una texture uniforme per l'intero blocco ora si potrà selezionare singolarmente una texture per ognuna delle facce.

Tutto è stato reso possibile aggiungendo un nuovo file "Blocks.hh".

Nuove aggiunte:

1. CLASSE GPUCUBE

```
class GPUcube
{
private:
    GLuint vbo;
    GLuint vao;
    GLuint ebo;
    Blocks::TextureArray textureArray;
```

In questa tappa viene introdotta la classe **GPUcube** che sarà la base della costruzione del nostro cubo e del rendering.

Contiene:

- **textureArray** (GLuint)
È un'istanza della classe `Blocks::TextureArray` che gestisce `GL_TEXTURE_2D_ARRAY`. Internamente memorizza una variabile di tipo **GLuint**.

2. BLOCKS.HH:

Aggiunto in questa tappa, contiene le fondamenta del sistema di gestione dei blocchi.

```
enum class BlockType : uint8_t
{
    AIR = 0,
    DIRT,
    GRASS,
    STONE,
    WOOD,
    LEAVES,
    GLASS
};

enum class BlockFace : uint8_t
{
    Front = 0, // +Z
    Back,     // -Z
    Left,     // -X
    Right,    // +X
    Top,      // +Y
    Bottom    // -Y
};

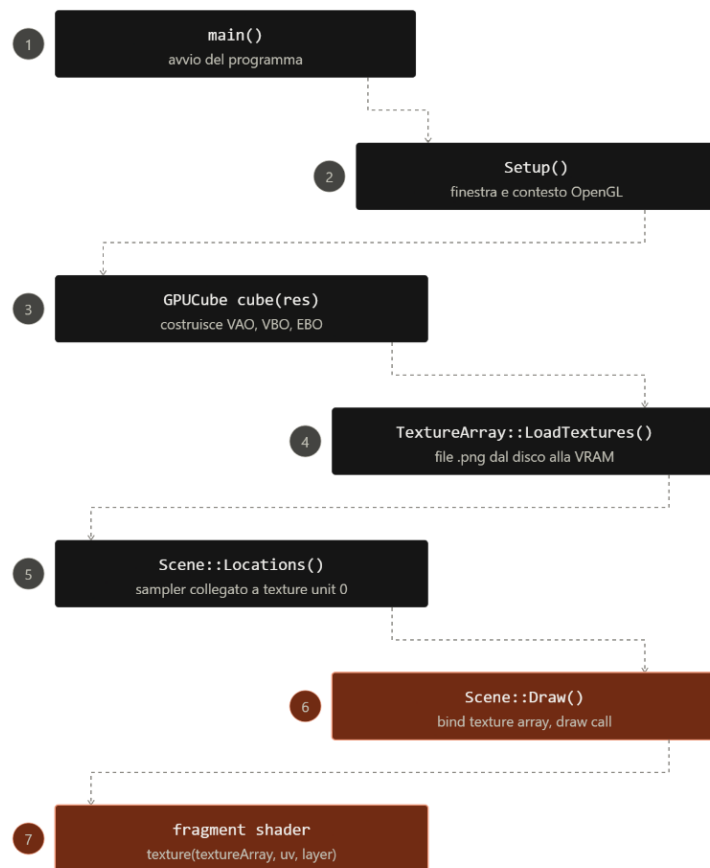
struct Block
{
    BlockType type = BlockType::AIR;

    bool isSolid() const{
        return type != BlockType::AIR;
    }
};
```

Il sistema distingue tra i vari blocchi tramite l'utilizzo di un enum "BlockType", dove sono elencati tutti i possibili tipi di blocchi potenzialmente visibili all'interno del gioco.

Un ulteriore enum "BlockFace", permette di dare **un'identità** a ciascuna faccia del cubo rendendo così possibile anche l'utilizzo di diverse texture e orientazioni differenti.

COME VIENE CARICATA E ASSEGNATA UNA TEXTURE?



Prima si caricano tutte le texture in memoria tramite la funzione

LoadTextures(...).

Qui viene creato un **"GL_TEXTURE_2D_ARRAY"** dove le texture saranno copiate e salvate.

Poi con l'utilizzo del metodo **getTextureIndex(...)** invece sarà possibile recuperare l'indice.

Attualmente la funzione **non è utilizzata** in questa tappa, infatti nella dichiarazione del cubo, tutti gli indici delle texture sono "hardcoded" per motivi di testing.

Principali Differenze:

- SHADER_FLAT.VERT E SHADER_FLAT.FRAG

Gli shaders hanno ricevuto delle modifiche.

```
#version 410 core

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec2 uvCoordinates;
layout (location = 2) in float textureIndex;

out vec2 outUvCoordinates;
out float outTextureIndex;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;
```

È stato aggiunto un nuovo attributo float, **"TextureIndex"**, che specifica quale texture prendere dal **GL_TEXTURE_2D_ARRAY**.

TAPPA03 – Creazione del concetto di Chunk e utilizzo di Mesh

Obiettivo:

La Tappa03 si propone come obiettivo l'introduzione il concetto di Chunk e Mesh.

CHE COS'È UN CHUNK?

Un chunk non è altro che una porzione, segmento o parte di mondo di grandezza arbitraria necessaria a distribuire e distinguere i vari blocchi presenti nel mondo voxel 3D in modo più chiaro e semplice.

In Minecraft un chunk è rappresentato da 16x256x16 blocchi (X,Y,Z). [pre-versione 1.18] [Minecraft Wiki](#)

Esempio da Minecraft, i bordi gialli rappresentano i confini del chunk:



TAPPA03

A partire dalla tappa03, il mondo non è più un singolo cubo disegnato a mano e “hardcoded”, ma una griglia [vector<chunk> + vector<blocks>] di porzioni di terreno generate proceduralmente, ciascuna con i propri blocchi e la propria mesh.

Questo sistema rende possibile la creazione di mondi **scalabili** e potenzialmente **infiniti**!

Il numero di blocchi visualizzabili nel mondo non è più limitato, aggiungendo altri chunk alla griglia si può far crescere quanto serve.

È stato aggiunto un nuovo file, “Chunk.hh”, che sostituisce completamente la classe **GPUCube** introdotta nella Tappa02.

GPUCube rappresentava un unico cubo con **vertici**, **UV** e **indici** di texture scritti a mano in un array statico, serviva un solo **VAO/VBO/EBO** fissato in fase di compilazione.

Chunk.hh sostituisce questa logica con una vera e propria struttura dati propria e le proprie funzioni di gestione e creazione del mondo.

Nuove aggiunte:

1. CLASSE CHUNK

Rappresenta una porzione di mondo di 16×64×16 blocchi (CHUNK_SIZE_X/Y/Z), memorizzati in un unico `std::vector<BlockType>` indicizzato linearmente.

```
namespace Blocks
{
    constexpr int CHUNK_SIZE_X = 16;
    constexpr int CHUNK_SIZE_Y = 64;
    constexpr int CHUNK_SIZE_Z = 16;

    class Chunk{
    private:
        std::vector<BlockType> blocks;
```

Fornisce:

Get/Set per leggere e modificare un blocco.

IsSolid per sapere se un blocco è pieno.

IsTransparent per sapere se lascia vedere gli altri blocchi attraverso di esso.

Se le **coordinate** richieste sono fuori dai bordi del chunk, Get restituisce semplicemente **AIR**: è così che il bordo del chunk risulta visibile finché non esiste ancora un chunk vicino.

2. BUILDCHUNKMESH E OTTIMIZZAZIONE

La mesh di un chunk non contengono tutte le facce di tutti i blocchi.

BuildChunkMesh scorre ogni blocco non **AIR** e, per ciascuna delle sue sei facce, genera i vertici solo se il blocco adiacente in quella direzione è trasparente.

In questo modo le facce completamente nascoste all'interno del terreno (es. sotto la superficie) non vengono nemmeno inviate alla GPU, risparmiando parecchi vertici.

3. GENERAZIONE PROCEDURALE DEL TERRENO

Per impostare il mondo 3D Voxel ho dovuto creare una prima versione della generazione procedurale del terreno.

La funzione **fillExistingChunks(...)** esegue questo compito, riempie ogni chunk per livelli:

- Erba e "Alberelli" in superficie.
- Terra semplice subito sotto.
- Pietra in profondità.

Fino ad arrivare al limite del mondo di gioco.

TAPPA04 – Crosshair e contorno blocchi

Obiettivo:

Questa tappa fa da introduzione alle meccaniche d'interazione tra il giocatore e il mondo voxel 3D.

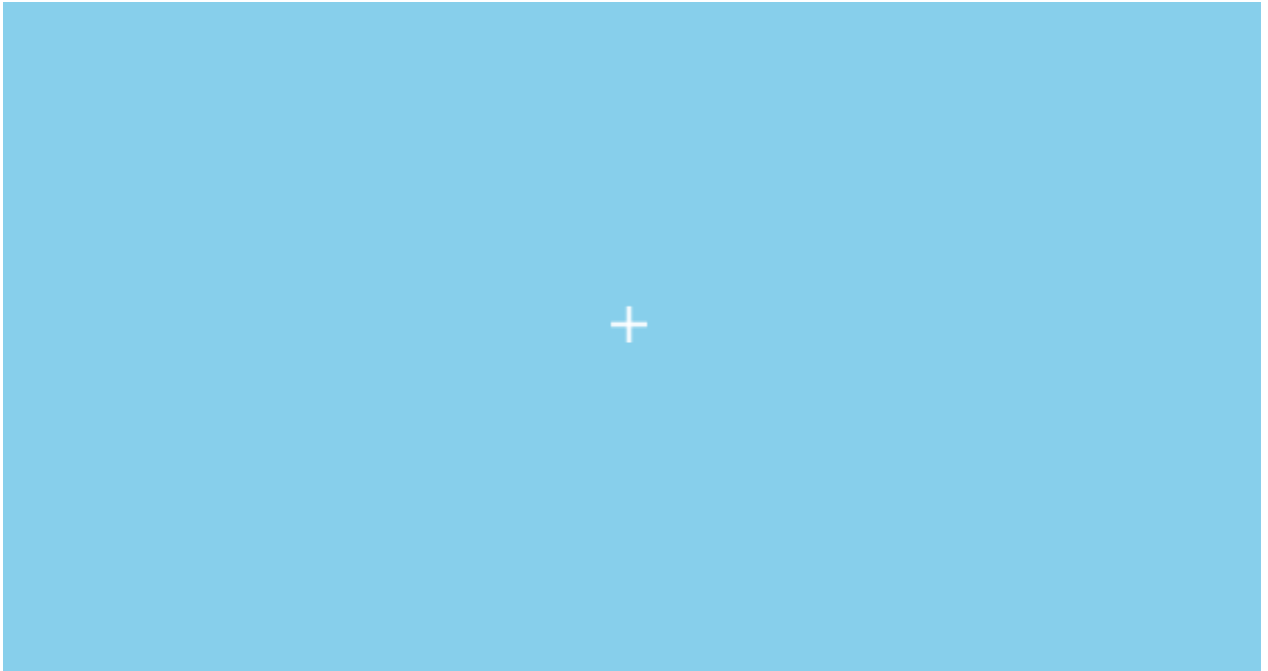
La Tappa04 introduce un **puntatore** "Crosshair" alla camera del mondo e un **contorno bianco** ai blocchi a cui essa punta.

Il giocatore può ora **individuare quale blocco sta guardando**, vederlo **evidenziato** da un contorno, e orientarsi con la crosshair. Un vero e proprio primo passo verso meccaniche più avanzate, come la rottura e piazzamento dei blocchi.

1. CROSSHAIR

La crosshair è stata implementata con usando **OpenGL**.

Renderizzazione della crosshair in gioco:



Due rettangoli (quad in questo caso), una barra orizzontale e una verticale, vengono generate e renderizzate al centro dello schermo per dare illusione dell'esistenza di questo puntatore fisico a forma di croce.

Per evitare problemi e farli renderizzare sempre davanti a noi il Depth test viene momentaneamente disattivato prima della chiamata della funzione di disegno Draw(...) utilizzando `glDisable(GL_DEPTH_TEST)`.

Per far funzionare tutto correttamente ho usato degli shader dedicati e separati dalla renderizzazione del mondo:

Shader_crosshair.vert/frag:

Il vertex shader corregge la distorsione da aspect ratio (`pos.x /= aspectRatio`).

Il fragment shader restituisce un colore bianco quasi opaco (`vec4(1,1,1,0.9)`).

2. CAMERA "CLICK AND DRAG" A STILE FPS

Inoltre, in questa tappa la camera ora si comporta in modo diverso...

Prima per modificare gli angoli di vista della camera (**YAW** e **PITCH**) era necessario tenere premuto il **tasto sinistro** del mouse, ora invece la camera si muoverà direttamente ai movimenti presi in input dal mouse senza avere la necessità di dover tenere premuto per guardarsi attorno.

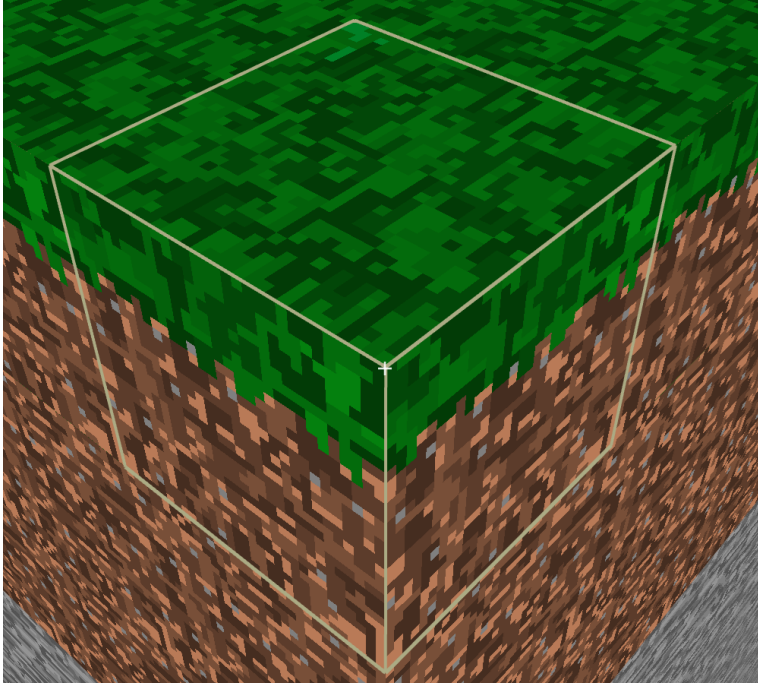
Il cursore viene nascosto e catturato dalla finestra, ricentrandolo ad ogni frame.

3. CONTORNO BLOCCHI

In questa versione è stata aggiunta anche una nuova funzionalità per far capire al giocatore il blocco a cui lui stia puntando.

È stato aggiunto un contorno bianco attorno che avvolge qualsiasi blocco attualmente puntato, questo è fondamentale per rendere il piazzamento e rompimento di blocchi più chiaro e reattivo.

Renderizzazione del contorno in gioco:



COME È STATO IMPLEMENTATO?

Nonostante sia una funzionalità all'apparenza semplice, in realtà ciò che ci sta dietro è complesso.

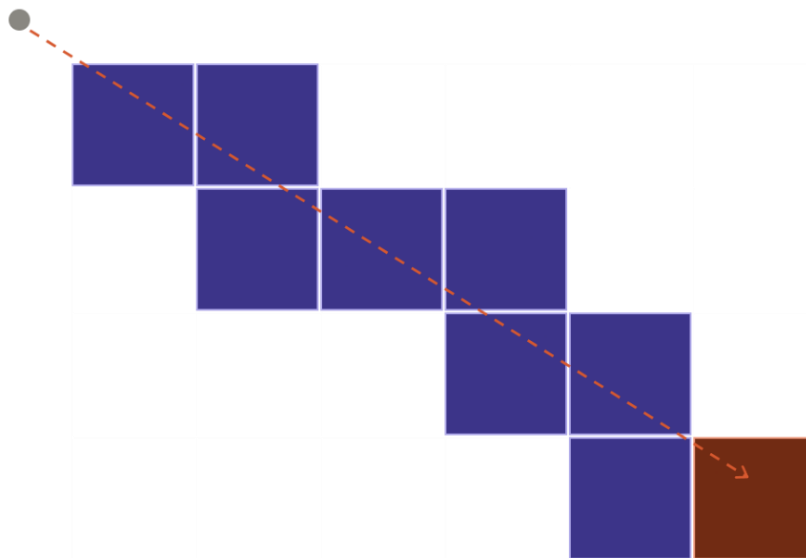
1. Come capiamo se la camera o giocatore in un punto nello spazio ha davanti un blocco?
2. Come facciamo a capire tra tutti i possibili blocchi quale sia quello a cui stiamo puntando?

Per risolvere questi due quesiti ho deciso di usare la tecnica del **raycasting**.

3.1. COS'È UN RAYCAST?

Un **raycast** è un laser o linea retta immaginaria, che da un punto di partenza A (la camera) va in una direzione B (dove la camera sta guardando), per scoprire se collide con qualcosa lungo il suo percorso.

Nel nostro caso la linea retta parte dalla posizione della camera e prosegue nella direzione in cui il giocatore sta guardando, fino a una distanza massima arbitraria. (Nel nostro caso 6.0f).



● Camera

■ Celle esaminate (vuote)

■ Blocco trovato (hit)

3.2. COME CAPIAMO SE C'È DAVANTI A NOI UN BLOCCO?

Per farlo in modo efficiente e preciso usiamo l'algoritmo **DDA (Digital Differential Analyzer)**, implementato in RaycastBlock. L'idea è questa:

- Invece di avanzare a passi fissi (che potrebbero "saltare" dei blocchi), calcoliamo ad ogni iterazione **quanto manca per raggiungere il prossimo confine del blocco più vicino**, separatamente su tutti gli assi **X, Y e Z**.
- Ad ogni passo scegliamo di **avanzare** sull'asse il cui confine è più vicino (quello con tMax più piccolo), e ci spostiamo esattamente nel blocco adiacente.
- Prima di avanzare, controlliamo se il blocco in cui ci troviamo non è **AIR** (funzione IsSolidAtWorld). Se **TRUE**: Quello è il blocco a cui stiamo puntando.
- Se il raggio supera la distanza massima senza trovare nulla, restituiamo **FALSE** "nessun blocco colpito".

In questo modo la retta attraversa **ogni singolo blocco della griglia esattamente una volta**, nell'ordine giusto, senza saltarne nessuno.

Il mondo voxel 3D non è continuo ma è diviso in una griglia di cubi di lato **1**.

Questo ci permette di semplificarci la vita, invece di controllare tutta la retta continuamente, possiamo avanzare controllando **un blocco della griglia alla volta**, e ad ogni blocco controlliamo se è pieno o è vuoto.

3.3. COME CAPIAMO QUALE BLOCCO STIAMO PUNTANDO?

Il DDA ci indica le coordinate dei blocchi che la retta attraversa.

Ma ad ogni blocco dobbiamo effettivamente controllare che esso sia presente.

Questo è il compito di IsSolidAtWorld(worldX, worldY, worldZ):

1. Controlla che worldY sia dentro i limiti verticali del mondo; se è fuori, è sicuramente **AIR**.
2. Calcola a quale chunk appartiene blocco calcolando ChunkX e ChunkY, con FloorDiv(worldX, CHUNK_SIZE_X) e FloorDiv(worldZ, CHUNK_SIZE_Z).
3. Cerca quel chunk con GetChunkAt. Se **non esiste** (siamo fuori dal mondo generato), la cella è considerata **AIR**.
4. Se il chunk **esiste**, converte le coordinate mondo in coordinate **locali** al chunk e chiama chunk->IsSolid(localX, worldY, localZ), che controlla semplicemente se il BlockType in quella cella è diverso da **AIR**.

Funzione isSolidAtWorld:

```
//Controlla se è solido il blocco da coordinate MONDO a locali.
bool IsSolidAtWorld(int worldX, int worldY, int worldZ){
    if(worldY < 0 || worldY >= Blocks::CHUNK_SIZE_Y){
        return false;
    }

    int chunkX = FloorDiv(worldX, Blocks::CHUNK_SIZE_X);
    int chunkZ = FloorDiv(worldZ, Blocks::CHUNK_SIZE_Z);

    Blocks::Chunk* chunk = GetChunkAt(chunkX, chunkZ);
    if(!chunk){
        return false;
    }

    int localX = worldX - chunkX * Blocks::CHUNK_SIZE_X;
    int localZ = worldZ - chunkZ * Blocks::CHUNK_SIZE_Z;

    return chunk->IsSolid(localX, worldY, localZ);
}
```

TAPPA05 – Rottura Blocchi

Obiettivo:

La Tappa05 espande l'interazione del giocatore con il mondo.

Introduce la possibilità di **rompere** il blocco selezionato con il tasto sinistro del mouse, aggiornando in tempo reale la mesh del chunk interessato (e degli eventuali chunk adiacenti).

Nuove aggiunte:

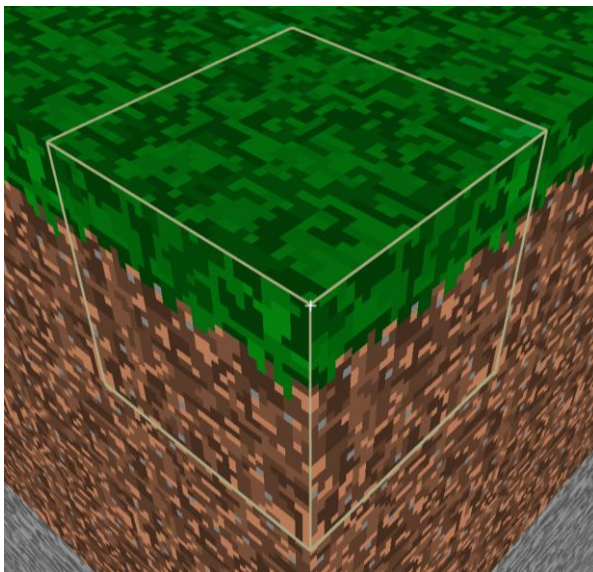
1. ROTTURA BLOCCHI

Con il tasto sinistro del mouse ora è possibile rompere i blocchi.

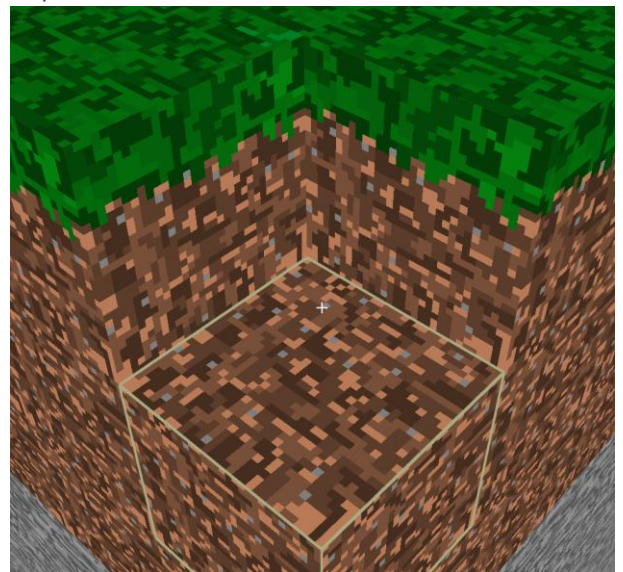
Riutilizzando il sistema del Raycast, ho creato **BreakBlockWorld**(worldX, worldY, worldZ):

- Individua il chunk di appartenenza a partire dalle coordinate mondo ricevute dal Raycast.
- Sovrascrive il tipo del blocco selezionato con il `BlockType::AIR`.
- Rigenera la mesh del chunk modificato.
- Se il blocco si trovava **su un bordo del chunk**, allora sarà necessario **rigenerare** anche la **mesh** dei chunk adiacenti (altrimenti resterebbe una faccia mancante o superflua sul confine).

Prima della rottura:



Dopo la rottura:



2. DIFFICOLTÀ RISCONTRATE

Facce mancanti sui bordi dopo una rottura:

Quando un blocco viene rimosso la mesh del chunk viene rigenerata, ma se il blocco rimosso era sul confine, allora si presenta un problema.

Le mesh dei chunk attaccati al nostro non vengono aggiornate, e quindi dato che il chunk vicino ha costruito la propria mesh assumendo quel blocco fosse ancora **solido**, **esso mostra ancora la faccia di quel blocco**.

Per risolvere questo problema si procede con la rigenerazione dei chunk adiacenti a quelli del blocco distrutto, **SOLO** quando la coordinata locale del blocco tocca il bordo (`localX == 0` o `localX == 15`), evitando di rigenerare tutto il mondo ad ogni rottura.

TAPPA06 – Piazzamento Blocchi

Obiettivo:

La Tappa06 completa questa parte di interazione del giocatore con il mondo.

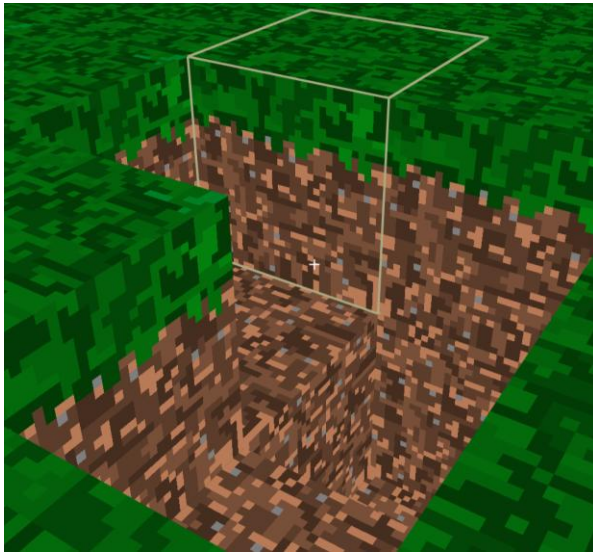
Introduce la possibilità di **piazzare** un blocco all'interno del mondo con il tasto destro del mouse, aggiornando in tempo reale la mesh del chunk interessato (e degli eventuali chunk adiacenti).

Nuove aggiunte:

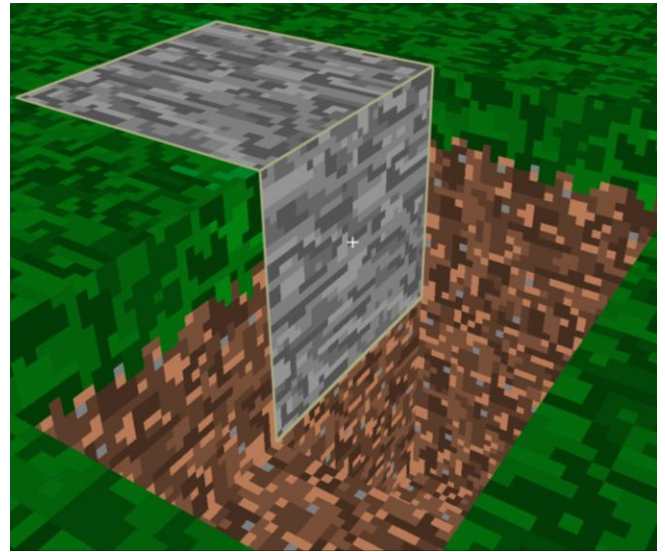
PIAZZAMENTO BLOCCHI

Ora con il tasto destro è possibile piazzare dei blocchi.

Prima del piazzamento:



Dopo il piazzamento:



COME È STATO IMPLEMENTATO?

Nonostante sia già presente la meccanica di rottura dei blocchi, qui c'è un altro problema da risolvere.

- Come sappiamo su quale faccia del blocco selezionato dev'essere piazzato il blocco che vogliamo piazzare?

Per poterlo implementare è stato necessario aggiornare e modificare il sistema del Raycasting.

Ci servono più informazioni! La posizione trovata dal Raycast non basta.

È necessario che teniamo conto anche della faccia del blocco colpito e trovato dal Raycast, così che quando andremo a piazzare un nuovo blocco si saprà esattamente dove il blocco deve andare finire.

Modifiche applicate a Raycast e alla funzione di calcolo del Raycast:

```
struct RaycastHit{
    bool hit = false;
    int blockX = 0;
    int blockY = 0;
    int blockZ = 0;
    int hitFace = -1; //Faccia colpita
};
```

```
if(tMaxX < tMaxY && tMaxX < tMaxZ){
    x += stepX;
    traveled = tMaxX;
    tMaxX += tDeltaX;
    enteredFace = stepX > 0 ? 2 : 3;
}
else if(tMaxY < tMaxZ){
    y += stepY;
    traveled = tMaxY;
    tMaxY += tDeltaY;
    enteredFace = stepY > 0 ? 5 : 4;
}
```


TAPPA07 – Gravità e Collisioni

Obiettivo:

La tappa07 espande ulteriormente l'interazione del giocatore con il mondo introducendo la **gravità** e le **collisioni**.

La camera "fluttuante" (volo libero senza vincoli) viene rimossa e sostituita con un vero corpo fisico per il giocatore che include:

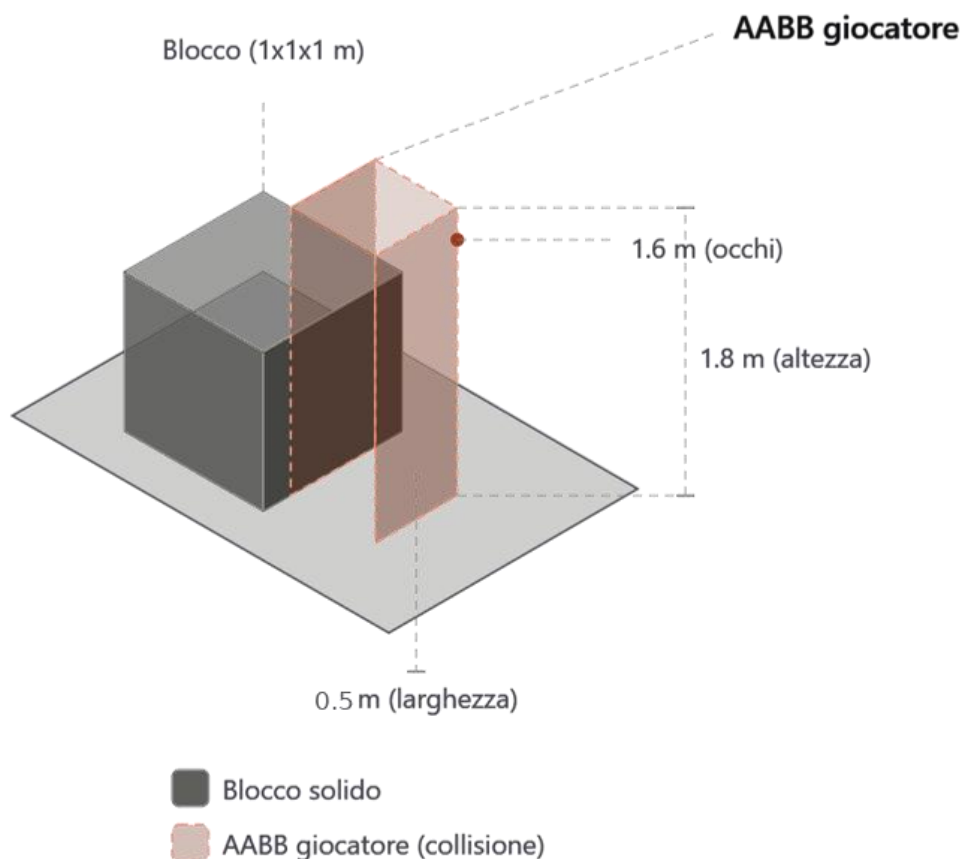
- Gravità .
- Collisioni contro il mondo voxel.
- Salto.

La modalità volo libero (**noclip**) è stata mantenuta solo per debug.

Nuove aggiunte:

1. COLLISIONE AABB ASSE PER ASSE

Il player è rappresentato come un **parallelepipedo** di larghezza 0.5f e altezza 1.8f.



Ad ogni frame lo spostamento viene applicato un asse alla volta (prima X, poi Y, poi Z):

Se dopo lo spostamento su un asse l'AABB interseca un blocco solido, quello spostamento viene annullato.

Muovere un asse alla volta evita che una collisione su un asse blocchi anche il movimento sugli altri (es. scivolare lungo un muro).

2. SICUREZZA SUL PIAZZAMENTO BLOCCHI

PlaceBlockWorld ora verifica `player.IsPlayerOccupyingBlock()` prima di piazzare un blocco, per evitare che il giocatore resti incastrato in un blocco appena posizionato.

3. CONTROLLIAMO SE IL GIOCATORE STA TOCCANDO IL TERRENO (ONGROUND)

Il controllo di onGround avviene in due momenti distinti:

- **Atterraggio**, mentre il player **cade**, velocity.y è negativa e delta.y con essa. Quando i piedi penetrano per la prima volta in un blocco solido, CollidesAt rileva la collisione direttamente; poiché $\text{delta.y} < 0$, si deduce che si trattava di una caduta e si imposta **onGround = true**.
- **A terra** (fermo o camminando senza saltare) la gravità non viene più applicata, quindi velocity.y resta a 0 e di conseguenza $\text{delta.y} = 0$ per ogni frame successivo. Con $\text{delta.y} = 0$ i piedi restano esattamente sul **confine** superiore del blocco, senza overlap. Per questo si usa una **piccola sonda** (**groundCheckEpsilon = 0.05f**) leggermente sotto i piedi, testata ad ogni frame in cui non c'è collisione diretta, indipendentemente dal valore di delta.y. Permette di confermare **onGround = true** in modo stabile, evitando che cambi costantemente tra true/false frame per frame (Con la conseguenza che il salto sia poco reattivo).

TAPPA08 – Refactor e Blocchi trasparenti

Obiettivo:

La Tappa08 non introduce nuove funzionalità di gioco.

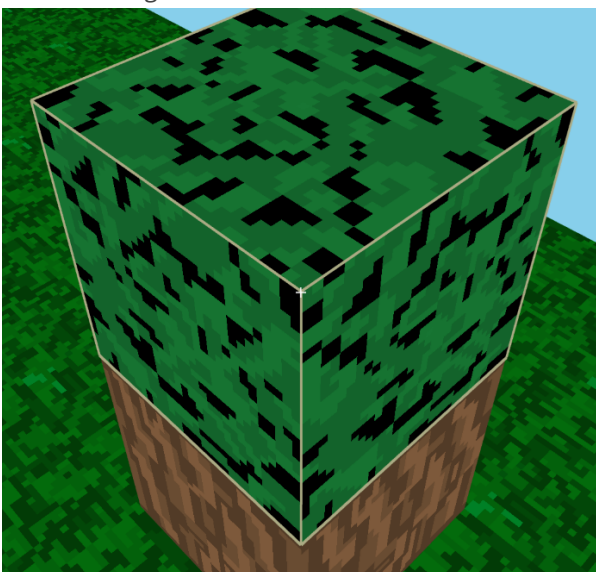
Il mondo, i comandi e l'aspetto sono identici alla Tappa07.

Gli obiettivi di questa tappa:

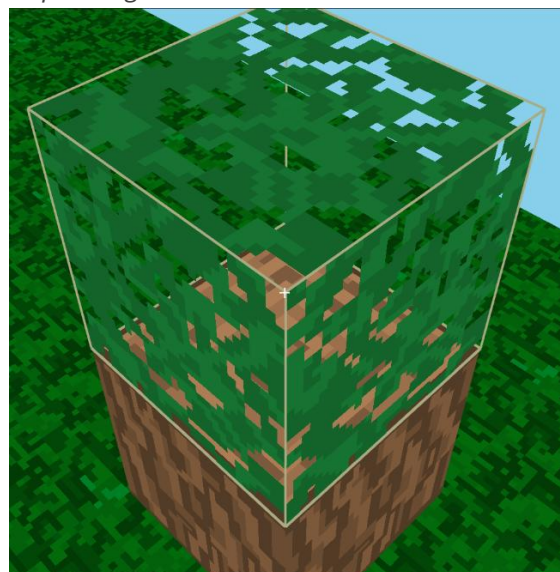
- **Correzione dei bug di rendering** relativi ai blocchi trasparenti (**LEAVES, GLASS**). [*shader_flat.frag*]
- **Re-fattorizzazione** dell'intero codice, che nella Tappa07 era quasi interamente presente in un unico file (NicoCraft.cc), separandolo in moduli (Camera, Player, World, Renderer), in preparazione delle tappe successive.

Prima di questa tappa tutti i blocchi che avevano dei pixel trasparenti (supportati dal formato .png) apparivano completamente neri; invece, da questa tappa in poi verranno correttamente renderizzati trasparenti, e sarà anche possibile vedere la faccia del blocco sottostante!

Prima del bugfix:



Dopo il bugfix:



TAPPA09 – Hotbar

Obiettivo:

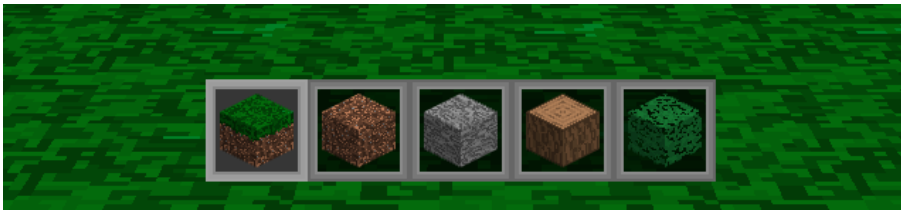
La Tappa09 introduce un nuovo modo d'interagire tra il mondo e il giocatore, la **hotbar**.

La hotbar indica e aggiunge la possibilità di poter interagire con più tipi di blocchi.

COME FUNZIONA?

Utilizzando i numeri [1-5] si può selezionare uno “slot”, e dipendentemente dallo slot selezionato una volta che l'utente tenterà di piazzare un blocco, esso sarà di quel tipo e non più solo del tipo **pietra** come nelle vecchie tappe.

Hotbar così come è vista in gioco:



Lo slot **selezionato** è indicato da una texture con un **bordo** leggermente più **chiaro** così che sia chiaro per il giocatore.

Per far capire all'utente che blocco andrà a piazzare, ho deciso di creare all'interno degli slot dei mini-cubi in stile isometrico, che con l'utilizzo delle texture dei blocchi stessi permettono subito di capire quale blocco sarà piazzato.

COME È STATA IMPLEMENTATA?

Per implementare la hotbar ho deciso di usare direttamente SFML, è stato quindi necessario passare da `sf::Window` a `sf::RenderWindow` nella classe Setup.

Perché solo la classe **RenderWindow** supporta le primitive 2D di SFML (`window.draw(vertices, ...)`) necessarie ad implementare la hotbar.

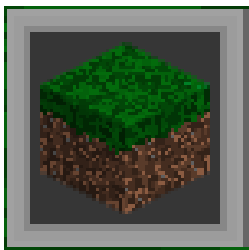
Inoltre, è stato necessario passare da un profilo `sf::Attribute::Core` a “Default” dato che le chiamate di SFML usano funzioni **deprecated** che OpenGL di norma non utilizza.

HOTBAR

La hotbar è divisa in due parti collegate da un'unica draw:

- Gli slot sono disegnati per primi, sono delle semplici coppie di triangoli e compongono un quadrato, e al di sopra di essi viene applicata la texture corrispondente, ovvero se lo slot è selezionato “hotbarSelected.png” oppure semplicemente “hotbarSlot.png” in caso contrario.

- I blocchi vengono creati da **tre facce** composte da triangoli e orientate correttamente, poi su di esse vengono applicate le texture corrette per il corrispettivo blocco che dovrebbero rappresentare.
Come vediamo **non** sono dei veri e propri **blocchi** ma sono dei triangoli 2D orientati in modo da dare l'illusione che sia 3D (Stile isometrico).
Inoltre, per dare l'illusione di profondità è stata aggiunta un po' di ombreggiatura modificando i valori dei colori dei vertici passati alla draw.



Per le ombre è bastato passare un valore diverso di color per ciascuna delle facce del cubo isometrico, che va a modificare la tinta della texture:

- Sopra - `sf::Color::White`.
- Sinistra - `sf::Color(140, 140, 140)`.
- Destra - `sf::Color(190, 190, 190)`.

TAPPA10 – Day/Night Cycle e Hotbar estesa

Obiettivo:

La Tappa10 introduce il ciclo giorno e notte, ed estende la scelta di blocchi della hotbar.

Questa tappa, inoltre, modifica anche il comportamento della **skybox** e della **luminosità** dei blocchi, che ora cambiano di colore e luminosità dipendentemente dall'ora del giorno.

1. HOTBAR ESTESA

Alla hotbar sono stati aggiunti alcuni blocchi che erano in via di sviluppo, il GLASS e le PLANK sono accessibili direttamente.

Hotbar estesa:



2. CICLO GIORNO/NOTTE

La parte più interessante di questa tappa di trova nella aggiunta del day/night cycle. Esso simula il passaggio del tempo nel mondo voxel.

Il tempo gestito da una singola variabile float, **elapsedTime**, che ad ogni frame viene incrementata di deltaTime (Sky::Update).

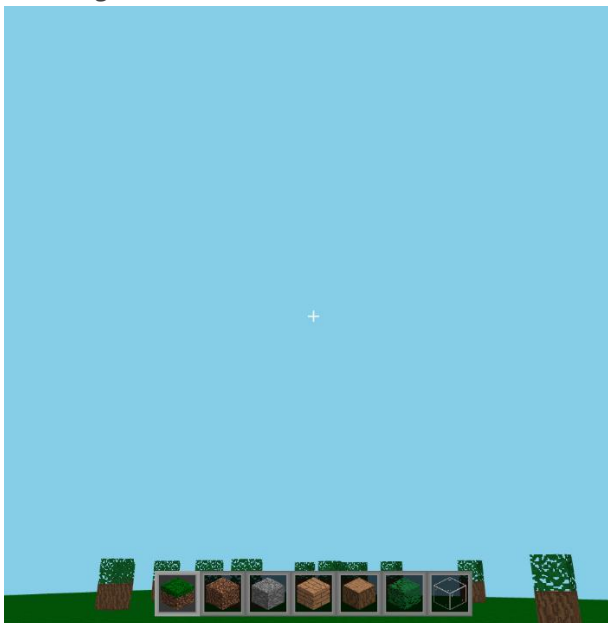
Da qui si ricava con una funzione matematica un valore T [0-1] che sarà utilizzato per:

- Cambiare il **colore** del cielo (interpolazione tra `nightSky` e `daySky`).
- Modificare la **luminosità** globale dei blocchi e l'alpha del sole, luna e stelle.

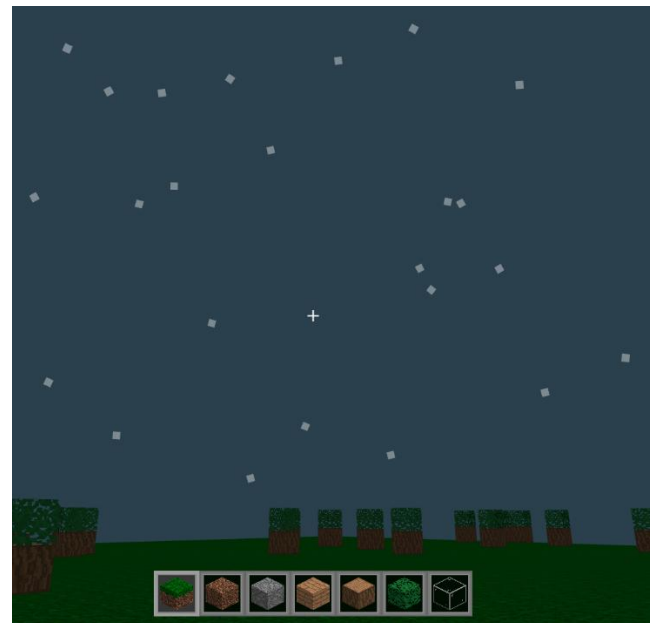
Il colore del cielo e il fattore di luce globale/ambientale si modificano gradualmente con il passare del **tempo** per dare l'illusione che ci sia un effettivo calcolo della luce dinamica.

La luminosità del blocco viene modificata direttamente all'interno di **shader_sky.frag**.

Cielo di giorno:



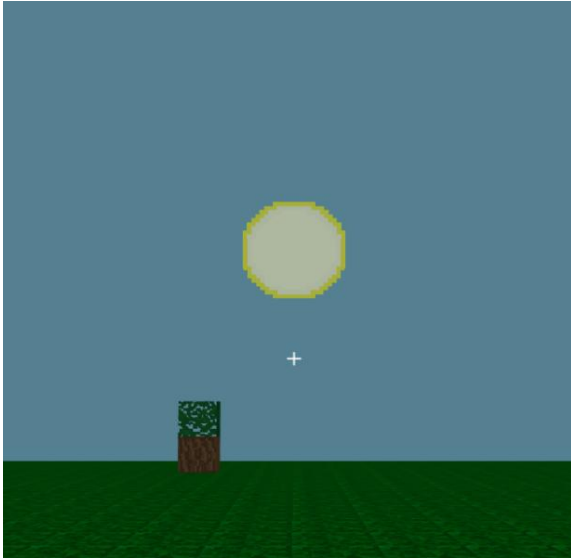
Cielo di notte:



IL SOLE E LA LUNA

Sono due dei quad texturizzati (un rettangolo da 2 triangoli, non 3D), disegnati con lo stesso VAO/VBO riutilizzato per entrambi, il buffer viene semplicemente riscritto ogni frame con `glBufferSubData` prima di ogni draw call. *[UpdateCelestialQuad]*

Il Sole:



La luna e stelle:



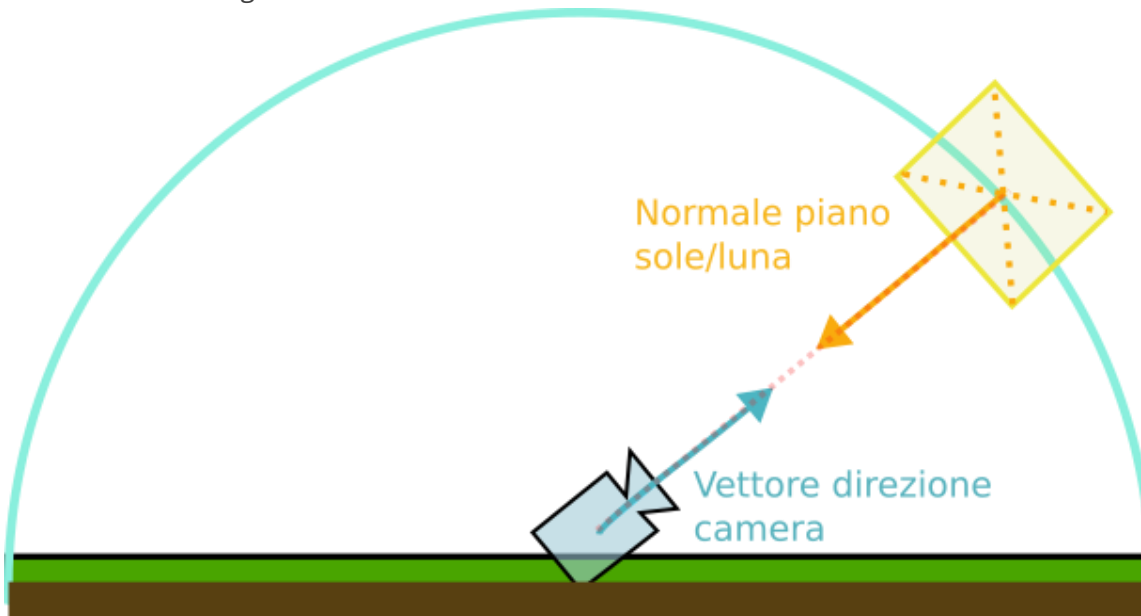
Nel mondo voxel 3D il sole e la luna ruotano in un arco **fisso** attorno al giocatore a una distanza **costante** dalla camera.

Il quad viene sempre visto "di fronte", orientato sempre ed esclusivamente verso il giocatore, qualunque cosa la camera stia guardando.

COME APPAIONO SEMPRE "DI FRONTE"?

Per far apparire sempre di fronte sia il sole che la luna utilizziamo la tecnica del "billboarding".

Tecnica billboarding:



Quindi la normale al piano del sole soddisfa due condizioni principali:

1. Deve avere stessa direzione ma verso opposto rispetto al vettore direzione della telecamera.
2. Essere sempre perpendicolare alla tangente del semicerchio.

In questo modo il sole sarà rivolto sempre verso il giocatore (ruotato correttamente), e seguirà sempre la traiettoria del semicerchio.

COME "SEGUONO" IL PLAYER?

La loro posizione è ricalcolata ogni frame con la seguente formula:

$$\text{Posizione} = \text{cameraPos} + \text{circlePosition} * \text{raggio}.$$

Non sono ancorati a un punto fisso del mondo, ma sono ancorati alla **camera stessa**, con un offset che dipende solo dall'ora del giorno, questo permette di seguire il giocatore dando l'illusione che siano molto lontani.

LE STELLE

Le stelle vengono formate e renderizzate in modo diverso.

Ogni stella è un piccolo quad (4 vertici) con una rotazione casuale propria, cosicché non sembrano tutte allineate allo stesso asse.

L'orientazione delle stelle rispetto al player è sempre frontale (Tecnica del billboard), come quella del sole e della luna

Ma qui, a differenza loro, l'orientamento del quad non è precalcolato sulla *CPU* ma viene estratto direttamente dalla view matrix ad ogni vertice, e successivamente mandato e calcolato all'interno della GPU (**shader_stars.vert**).

La Tecnica del billboarding (*shader_stars.vert*):

```
//Assi camera estratti dalla view matrix, per il billboard verso lo schermo
vec3 cameraRight = vec3(view[0][0], view[1][0], view[2][0]);
vec3 cameraUp    = vec3(view[0][1], view[1][1], view[2][1]);
```

Questa è la tecnica classica dello "screen-facing billboard"...

Le colonne della view matrix contengono gli assi della camera in coordinate mondo, quindi si può costruire un piano sempre perpendicolare alla direzione di vista senza calcoli trigonometrici aggiuntivi.

Ogni stella viene poi posizionata con la stessa formula:

$$\text{Posizione Stella} = \text{cameraPos} + \text{direzione} * \text{starRadius}$$

Esattamente come Sole e Luna, stesso principio di "seguono il player restando a distanza fissa".

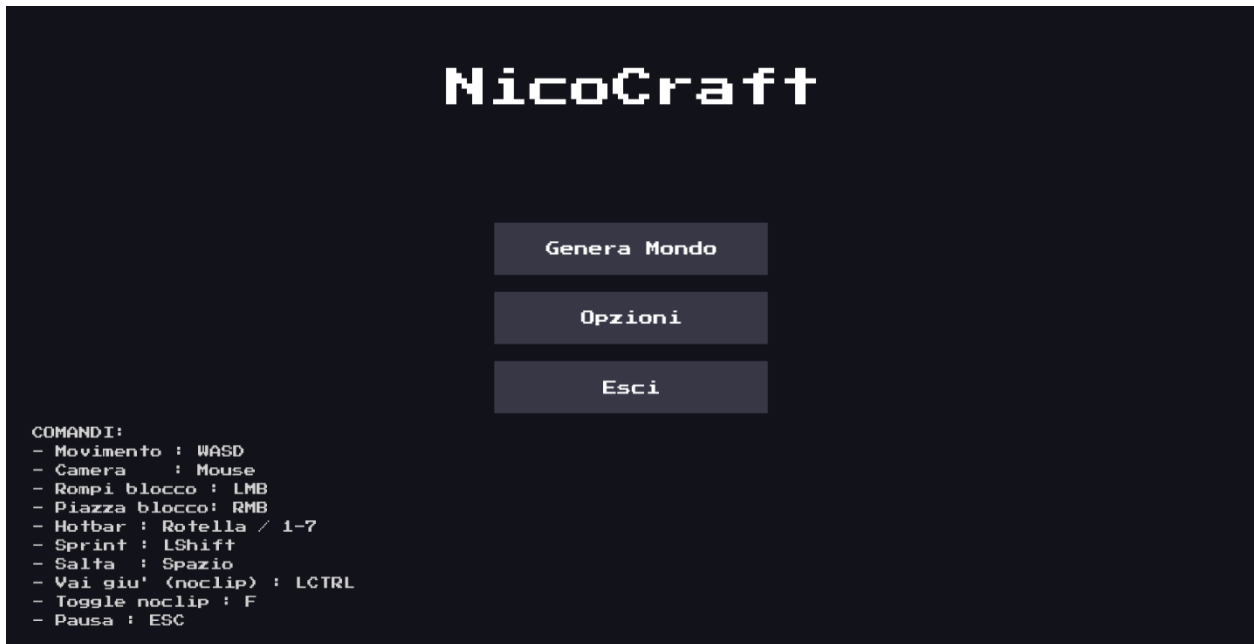
TAPPA11 – Menu Principale e ritocchi finali

Obiettivo:

La Tappa11 introduce il Menù principale, la possibilità di mettere in pausa il gioco e alcuni ritocchi.

MENÙ PRINCIPALE:

In questa tappa è stato aggiunto il menu principale, da cui ora si potrà generare il mondo e accedere anche alla schermata delle opzioni.



Inoltre, in basso a destra sono segnati tutti i comandi che potranno essere utilizzati dal giocatore all'interno del mondo voxel 3D.

PAUSA:

Un'altra aggiunta è la possibilità di mettere in pausa il gioco quando si è nel mondo voxel 3D.

Ora quando si preme il tasto "ESC" il gioco verrà messo in "pausa", a differenza di prima che invece si terminava l'esecuzione del programma.



Mentre il gioco è in pausa non sarà possibile muoversi o interagire con il mondo in alcun modo; infatti, esso non prenderà alcun tipo di input da parte dell'utente e il tempo T si fermerà.

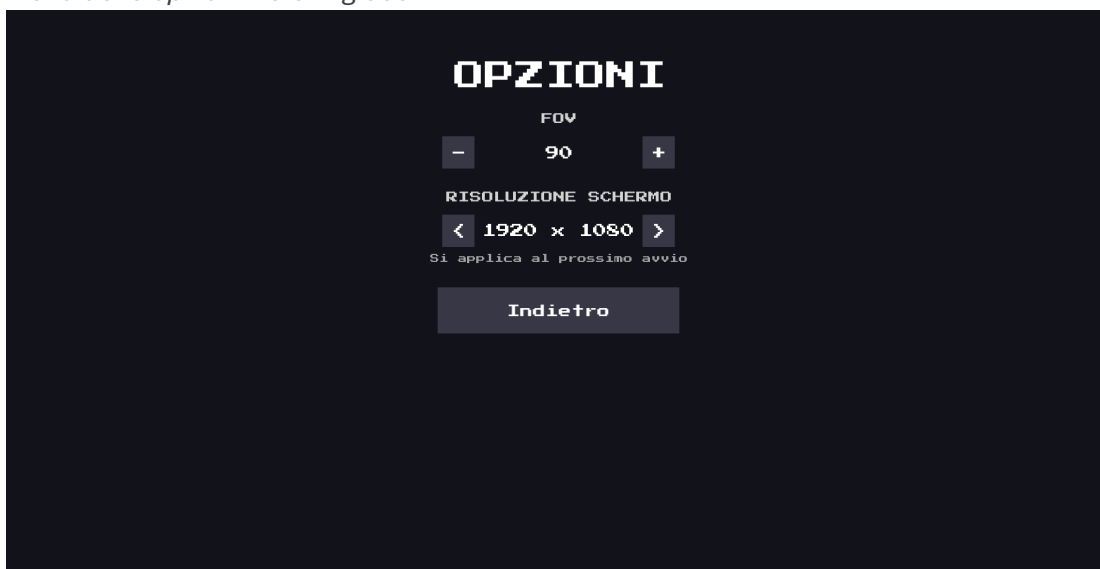
L'unico input accettato sarà il tasto sinistro del mouse, con cui si potrà interagire con i pulsanti disponibili:

- Ritorna al gioco
- Opzioni
- Menù Principale
- **Esci dal gioco** => Terminazione programma

LE OPZIONI:

Dal menu delle opzioni si potrà modificare sia la grandezza dello schermo, e il FOV del giocatore all'interno del mondo voxel 3D.

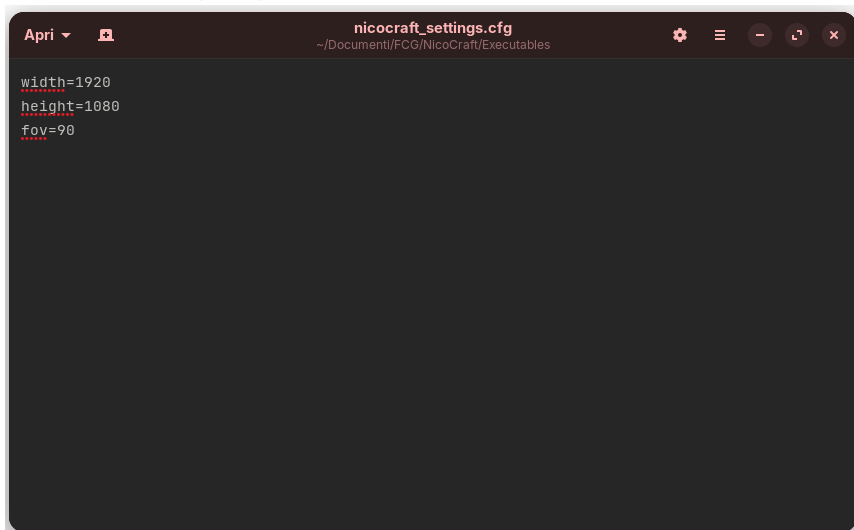
Menu delle opzioni visto in gioco:



- Il FOV verrà modificato e applicato direttamente al gameplay.
- Invece per modificare la risoluzione dello schermo sarà necessario un riavvio.

Per non perdere la risoluzione selezionata dall'utente le opzioni quando vengono modificate saranno salvate sul un file su disco, chiamato **“nicocraft_settings.cfg”**.

nicocraft_settings.cfg:



Ad ogni avvio si andrà prima a cercare il file “.cfg” con le opzioni e poi aprirà una finestra con le impostazioni desiderate.